

Universidade Federal do Rio Grande do Norte
Sistemas Biga Data DIM0198 - 2026.2

Relatório de Implementação de DBSCAN em MPI

Eduardo Nestor Ferreira Marinho
Kael Amir Pina Paraguassu Silva
Matheus Leão de Carvalho
Samuel Morais da Rocha e Silva

Natal, 3 de Novembro de 2026.

1 Implementação

1.1 DBScan

A implementação do algoritmo DBSCAN foi dividida em duas fases principais: o mapeamento de vizinhança e a expansão dos agrupamentos (*clusters*).

Na primeira etapa (`calcular_vizinhos`), o algoritmo mede a distância euclidiana entre os pontos em d dimensões e guarda quem está a no máximo ε (`eps`). O próprio ponto entra na lista. Para otimizar o processamento e facilitar a comunicação em rede, os vizinhos identificados para cada ponto são agrupados em um único vetor contíguo na memória (`plano`), em vez de serem armazenados em listas encadeadas independentes. Além disso, o vetor é acompanhado por um outro vetor auxiliar que registra a quantidade de vizinhos de cada ponto (`contagens`).

Na segunda etapa (`expandir`), os pontos são classificados a partir da densidade de sua vizinhança. Todos os registros iniciam rotulados como ruído (`-1`). O algoritmo então percorre a base em busca de pontos centrais, os que têm ao menos `min_pts` vizinhos nessa lista. Sempre que acha um ponto central ainda sem rótulo, abre um agrupamento e o expande em largura: vizinho de ponto central entra no grupo e, se também for central, a busca continua a partir dele.

1.2 Distribuição

Como a busca de vizinhos compara cada ponto com toda a base, o MPI divide essa carga em fatias. Todos os processos calculam a sua parte, inclusive o de *rank* 0:

1. **Leitura e Difusão (*Broadcast*):** O processo de *rank* 0 carrega o arquivo binário com a matriz normalizada e envia as dimensões e a matriz inteira aos demais com `MPI_Bcast`.

2. **Particionamento e Busca Paralela:** O conjunto total de n pontos é dividido de forma balanceada em fatias contíguas entre os processos disponíveis. Cada processo calcula, de maneira independente e sem necessidade de comunicação intermediária, as listas de vizinhos apenas para os pontos pertencentes à sua fatia local.

3. **Consolidação (*Gather*):** Os processos trocam o tamanho das listas (`MPI_Allgather`) e devolvem as contagens e os vizinhos ao processo 0 com `MPI_Gatherv`.

4. **Expansão Sequencial:** Com as listas reunidas, só o processo 0 executa `expandir`. O resultado não depende do número de processos, porque essa etapa é sempre a mesma.

O monitoramento de desempenho é realizado com barreiras de sincronização (`MPI_Barrier`) e cronometragem (`MPI_Wtime`), isolando os tempos gastos na busca paralela (consolidado pelo maior tempo entre os processos via `MPI_Reduce`), na junção dos dados pela rede e na expansão final.

1.3 Job

Os testes de desempenho e escalabilidade foram automatizados no gerenciador de filas Slurm por meio do script `job_dbscan.sh`. O ambiente de execução foi dimensionado na

partição `intel-128` com a alocação de 16 nós computacionais e até 32 tarefas por nó (1 CPU por tarefa), totalizando até 512 processos simultâneos e tempo limite de 30 minutos.

A bateria experimental executou o algoritmo sobre o arquivo `dados/amostra.bin` utilizando os parâmetros fixos $\varepsilon = 0{,}035$ e MinPts $= 30$. Para avaliar o comportamento do código em diferentes escalas de paralelismo, o número de processos (P) foi variado em potências de dois ($1, 2, 4, 8, 16, 32, 64, 128, 256$ e 512), realizando-se 3 repetições consecutivas para cada configuração a fim de obter as médias.

1.4 Saídas

Ao término de cada execução, o processo mestre consolida as métricas e gera os artefatos de saída na pasta `resultados/`:

- **Métricas de Desempenho (`tempos.csv`):** Arquivo tabular incrementado a cada execução com o número de processos, o índice da repetição, o tempo total, os tempos parciais de cada etapa (busca, junção e expansão) e os totais de agrupamentos e pontos de ruído.
- **Rótulos de Agrupamento (`rotulos_p<nprocs>.bin`):** Arquivo binário contendo o vetor de classes atribuídas a cada um dos n pontos da amostra, utilizado posteriormente pelos scripts de visualização.
- **Resumo da Execução (`resumo.json`):** Gerado na execução de referência com 1 processo, documenta os hiperparâmetros utilizados, a quantidade de agrupamentos e ruídos encontrados e o tamanho dos 10 maiores *clusters*.

2 Resultados

2.1 Desempenho

2.1.1 Eficiência, *speedup* e tempos de execução

A tabela pura produzida em formato CSV foi agregada com o objetivo de resumir as métricas de tempo de execução em relação às três execuções para cada configuração de número de processos. O resultado é uma tabela com os seguintes atributos:

- N_p (Índice): O número de processos paralelos utilizados na execução do algoritmo.
- $\bar{T}_{total}$: O tempo médio total decorrido para a execução completa do algoritmo considerando as repetições (em segundos).
- $\bar{T}_{buscar}$: A média do maior tempo gasto na etapa de busca de vizinhos dentro do algoritmo (em segundos). Utilizamos o maior tempo para a operação de busca entre os nós por este ser o gargalo da execução distribuída.
- $\bar{T}_{gather}$: O tempo médio gasto no processo de consolidação de dados paralelos (operação *gather*) (em segundos).
- $\bar{T}_{expandir}$: O tempo médio gasto especificamente na expansão dos clusters paralelos (em segundos).

Tabela 1: Métricas médias de desempenho por número de processos paralelo.

N_p	$\bar{T}_{total}$	$\bar{T}_{buscar}$	$\bar{T}_{gather}$	$\bar{T}_{expandir}$
1	1.2850	1.2813	0.0023	0.0014
2	0.6912	0.6871	0.0021	0.0014
4	0.3313	0.3252	0.0035	0.0014
8	0.1891	0.1808	0.0044	0.0017
16	0.1038	0.0920	0.0058	0.0015
32	0.1352	0.1010	0.0165	0.0020
64	0.0962	0.0532	0.0111	0.0018
128	0.1114	0.0291	0.0019	0.0019
256	1.1330	0.0160	1.1086	0.0021
512	1.1218	0.0084	0.9497	0.0023

Tabela 2: *Speedup* e Eficiência médias obtidas para o tempo total de execução.

N_p	$\bar{S}$	$\bar{E}$
1	1.0000	1.0000
2	1.8591	0.9296
4	3.8791	0.9698
8	6.7955	0.8494
16	12.3796	0.7737
32	9.5066	0.2971
64	13.3548	0.2087
128	11.5318	0.0901
256	1.1342	0.0044
512	1.1455	0.0022

A Tabela 1 consolida os resultados obtidos.

De posse da agregação, foi construído um gráfico de linhas da Figura 1 detalhando, para cada número de processos, o tempo médio total da execução da rotina e das operações que a compõe.

Dentre pontos de observação valiosos do gráfico, destacam-se:

- O salto no tempo decorrido para a operação de *gather* mediante o aumento de 128 a 256 processos. O salto é justificado pelo uso da partição `Intel-128` para a execução do *job*: sua limitação de 128 núcleos faz com que a partir deste ponto, a comunicação entre nós computacionais distintos seja inevitável. Essa dominância da latência de rede sobre o tempo de execução é expressado pelo aumento no tempo dedicado à operação *gather*.

- O tempo decorrido para a operação de busca decai de forma quase linear com a proliferação dos processos. Isso ocorre pois esta operação é **local**, não envolvendo trocas de mensagem na rede. Ou seja, esta operação beneficia-se grandemente pelo processamento distribuído, já que a quantidade de pontos processados em cada instância reduz com a adição de processos, diminuindo, também, o esforço de cada processador.

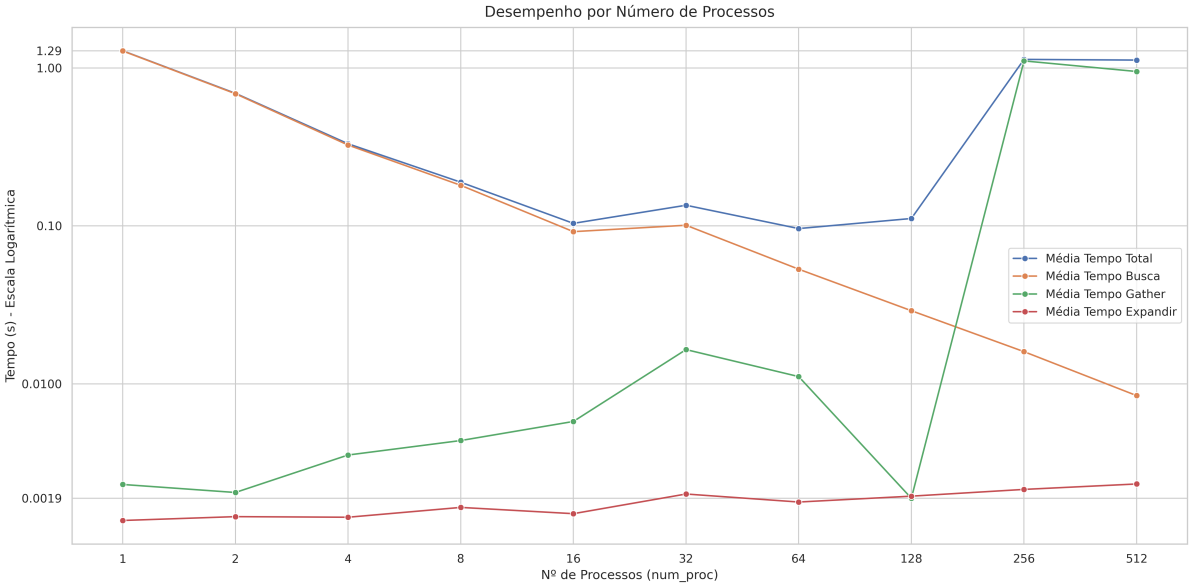


Figura 1: Gráfico de tempo de execução do DBScan.

- O tempo decorrido para a operação de busca decai de forma quase linear com a proliferação dos processos. Isso ocorre pois esta operação é **local**, não envolvendo trocas de mensagem na rede. Ou seja, esta operação beneficia-se grandemente pelo processamento distribuído, já que a quantidade de pontos processados em cada instância reduz com a adição de processos, diminuindo, também, o esforço de cada-processador.

2.2 *Clustering*